

A Voyage to the Kernel



Part 13

Segment 3.2: Day 12

In the last column we focused on some of the fundamentals in kernel programming. We are going to dedicate today's voyage to reviewing some more basic literature.

As I mentioned before, present day CPUs can run in two modes -- kernel mode and user mode. Interrupt drivers and operating system services are examples of those that can run in kernel mode. In this mode, free access to the entire memory and device registers are supported with the help of an 'extended set' of instructions. While when the CPU runs in user mode, it will have access only to a specific restricted set of instructions (the CPU cannot use the entire memory in this case). These two modes are used for security reasons. This, in turn, provides reliability to the operating systems. Normally, a program uses the user mode (which is 'safer'). When it needs more utilities and restricted resources, it will switch over to the kernel mode. Systems calls are used to access kernel utilities and are essentially operating system services. These software interrupts are processed by the OS as kernel mode processes.

We can see a system call list maintained by the OS and it has respective pointers to the functions that can implement the calls in the kernel. Here, you can see a list of such system calls (from the *syscall.h* file):

```
#ifndef _SYSCALL_H
#define _SYSCALL_H 1
```

```
/* This file should list the numbers of the system the system
   knows.
```

```
    But instead of duplicating this we use the information
    available
    from the kernel sources. */
```

```
#include <asm/unistd.h>
```

```
#ifndef _LIBC
```

```
/* The Linux kernel header file defines macros '__NR_<name>',
   but some
```

```
   programs expect the traditional form 'SYS_<name>'. So in
   building libc
```

```
   we scan the kernel's list and produce <bits/syscall.h> with
   macros for
```

```
   all the 'SYS_.' names. */
```

```
#include <bits/syscall.h>
```

```
#endif
```

```
#endif
```

This code is taken from the GNU C library. Here, you can see a dependency file. And you may find some of those included in the sidebar on 'syscalls'.

OS services can be well accessed using these calls. In the last column, we discussed the module programming part. There we stumbled upon two types of modules—essential and loadable. As the name suggests, the loadable ones can be loaded (or unloaded) based on the needs of the user (or program). These modules can provide extra utilities and modes to the kernel. As most of the readers would be aware of the basic differences between a micro kernel and a monolithic one, you can understand why some of them are arguing for micro kernel model!

If a new functionality offered by the 'module' has to be added directly to the original code, then it will be tedious, as you need to rebuild it every time you add a new portion of 'extra code'!